

NeuroForge — Team Setup Guide (Docker + CI/CD)

Follow these steps in order — each one depends on the one before it. Steps 1–3 get the app running locally; steps 4–5 connect a project to GitHub Actions so Trigger/Rollback work.

1 Clone / pull the repo

```
git clone <repo-url>
cd <repo-folder>

# or, if you already have it:
git pull origin main
```

2 Create your local `.env` file

This file is **git-ignored** — each teammate creates their own, it is never committed. Create `.env` in the project root (same folder as `docker-compose.yml`):

```
# Database – use the SAME value you'll put in the GitHub repo's
# DB_PASSWORD secret in step 5, so local and CI stay in sync
DB_PASSWORD=Jashan@04

# App-level encryption for stored GitHub tokens (project_integrations table)
# Any long random string works locally – just keep it the same across
# restarts, or saved GitHub connections will fail to decrypt
APP_ENCRYPTION_SECRET=some-long-random-string-change-me
APP_ENCRYPTION_SALT=0ecfd0a25de4e1a403a19c18899a0222
```

That's it — three variables total. GitHub credentials are **not** set here anymore; they're connected per-project from inside the app in step 4.

Double-check `.env` is listed in `.gitignore`. If it isn't, add it now — committing these values would leak them into git history. `APP_ENCRYPTION_SALT` must be a hex string (generate a fresh one with `openssl rand -hex 16` if needed).

3 Run the project locally

```
docker compose up --build
```

This starts, in order (each waits on the previous being healthy):

postgres → keycloak → backend → frontend

First boot is slow — Keycloak's health check alone can take a few minutes. Once up:

- Frontend: `http://localhost:5173`
- Backend API: `http://localhost:9000`
- Keycloak admin: `http://localhost:8080` (login: `admin` / `admin`)

If the backend fails to start immediately, it's almost always a missing/incomplete `.env` — recheck step 2 before debugging code.

4 Connect a project to GitHub (in-app)

Each **project** in NeuroForge connects its own GitHub repository from that project's own Settings page. This is what powers the **Trigger build** and **Rollback** buttons on that project's Pipeline dashboard.

1. Open the project → **Settings** tab → **GitHub Repository** panel.
2. Fill in **Repo owner**, **Repo name**, **Branch** (defaults to `main`), and **Workflow file** (defaults to `ci-cd.yml`).
3. Generate a fine-grained Personal Access Token for the repo:
 - `github.com/settings/tokens` → Fine-grained tokens → Generate new token
 - Repository access → Only select repositories → choose the repo
 - Permissions → Repository permissions → **Actions** → **Read and write**
 - Generate → copy the token immediately (GitHub only shows it once)
4. Paste the token into the **Personal access token** field and click **Connect repository**. It's encrypted before it's stored and never returned by the API afterward.

5. Once connected, the panel shows a generated **Webhook secret** — copy it, you'll need it in step 5.

You need write access to the repo to create a token with Actions permissions — ping whoever owns the repo if you don't have that yet. Only Admins and Project Managers can open/edit this panel.

5 Add the matching values on GitHub

On that repo → **Settings** → **Secrets and variables** → **Actions**, add exactly these:

Repository secrets

Name	Value
CONTROLLER_URL	Full URL of the backend's webhook endpoint, e.g. <code>https://<your-tunnel>.ngrok-free.app/api/pipelines/webhook</code> .
DB_PASSWORD	Same value you used in your local <code>.env</code> in step 2 (used by the throwaway Postgres service container the workflow spins up for tests).
WEBHOOK_SECRET	The value copied from that project's Settings → GitHub Repository panel at the end of step 4. The workflow signs its results with this; the backend rejects the webhook if the signature doesn't match.

Repository variables

Name	Value
PROJECT_ID	The numeric ID of this project inside NeuroForge (visible on the project's Settings → Overview card, or in its URL: <code>/projects/<id>/...</code>). The workflow sends this as <code>projectId</code> so the build/deploy gets attached to the right project.
DEPLOY_ENV OPTIONAL	Default deployment environment for plain <code>git push</code> triggers (e.g. <code>STAGING</code>). Falls back to <code>STAGING</code> automatically if you don't set it.

Nothing else needs to be added manually — `GITHUB_TOKEN` is provided automatically by GitHub Actions on every run (used to push images to GHCR), and repo → **Settings** → **Actions** → **General** → **Workflow permissions** should already be set to **Read and write permissions** at the repo level.

Known temporary limitation: nothing is deployed yet, so `CONTROLLER_URL` currently has to point at whichever teammate's local machine is running the backend + an active `ngrok http 9000` tunnel. Webhook delivery only works while that tunnel is running and the secret is current. Coordinate who's "hosting" the controller before testing Trigger/Rollback end-to-end, and update `CONTROLLER_URL` whenever the tunnel restarts (free ngrok URLs rotate).

Quick troubleshooting checklist

Symptom	Likely cause
Backend won't start	<code>.env</code> missing or incomplete — needs <code>DB_PASSWORD</code> , <code>APP_ENCRYPTION_SECRET</code> , <code>APP_ENCRYPTION_SALT</code> (step 2)
Can't save the GitHub Repository form in Settings	Not signed in as Admin/Project Manager, or the token was left blank on first connect
Trigger/Rollback button fails silently	No repo connected for this project yet, or the saved PAT is expired / lacks Actions read-and-write permission
Webhook POST from Actions gets a 401	<code>WEBHOOK_SECRET</code> on GitHub doesn't match the current value in Settings — someone may have regenerated it without updating GitHub
Webhook POST from Actions gets a 404 “no GitHub repository connected”	<code>PROJECT_ID</code> variable doesn't match a real project ID, or that project's integration was disconnected
Dashboard never updates after a build	<code>CONTROLLER_URL</code> tunnel isn't running / has rotated (ngrok free URLs change on restart)
<code>docker compose up</code> hangs on Keycloak	Normal — its health check window is long (up to 10 min) on first boot